

FANTOM-II

I. FEATURES

FANTOM-II is a 1-K byte loader/monitor/debugger program for the 6800. It can be supplied in a ROM or 2708 EROM, or as firmware on the WINCE Control Module. FANTOM-II supports the following commands entered from a teletype or equivalent terminal such as the WINTEK Model BRB video terminal or the Texas Instrument ASR 700 (Silent 700).

- M - Display/Change Memory
- I - Display/Change Instruction
- R - Display MPU Registers
- CNTL-A - Display/Change Accumulator A
- CNTL-B - Display/Change Accumulator B
- CNTL-C - Display/Change Condition Codes Register
- CNTL-P - Display/Change Program Counter
- CNTL-X - Display/Change Index Register
- G - Go To User Program *Return to address*
- S - Single Step User Program
- D - Dump Memory
- P - Punch Loader Compatible Tape
- L - Load Memory

FANTOM-II also supports

- free-format hexadecimal input
- insertion and deletion of breakpoints
- interrupt vectors
- program transparent operation

Free-format means that leading zeros can be omitted when entering hexadecimal memory addresses, data, OP codes, or the first or second byte of an operand. For example, to enter address 00A0, the following are equivalent: 00A0; 0A0; A0. To enter the data word 02 we can type either 02 or 2. Further details are given in section XIV.

Listed below are some symbols and their meaning in this document

- CR - carriage return or carriage return key on terminal
- CNTL - control key on terminal
- * - FANTOM-II prompt indicating that it is in the command mode ready to accept a command
- LF - line feed or line feed key on terminal
- ESC - escape key on terminal
- C - Condition Code Register
- B - B Accumulator
- X - Index Register
- P - Program Counter
- S - Stack Pointer

II. RESET

Push the reset switch to return system control to FANTOM-II.

Pushing the reset switch reverts system control to FANTOM-II which responds with a CR (carriage return), LF (line feed) and an * (asterisk). The *

Indicates that FANTOM-II is in the command mode ready to accept any of the FANTOM-II commands. RESET should be used whenever power is first applied, or anytime the user wants to return system control to FANTOM-II, e.g., after a program crash.

III. DISPLAY MPU REGISTERS

Type R

To display the contents of the MPU registers type R. FANTOM-II will respond by printing the registers contents followed by a CR and * indicating return to the command mode. The following example illustrates the R command:

```
♦ R C=F0 B=BF A=10 X=0BA4 P=0000 S=EDC4
```

The MPU registers are used both by FANTOM and the users program. When FANTOM is controlling the system the current user values for the MPU registers are stored on the user's stack. Hence, the R command actually displays the content of these memory locations.

IV. MEMORY DISPLAY/CHANGE

Type M followed by the memory address

To display the content of any memory location type M followed by the hexadecimal address to that memory location. For example, to display the content of memory location 0001 start by typing M (FANTOM-II will respond by typing a space); then type 0001, or 001, or 01, or 1 followed by CR. FANTOM-II will respond by printing the memory location, followed by a space, followed by the content, e.g., 0001 03. To display the content of the next memory location type CR. FANTOM-II will respond with 0002 7E (assuming 7E is the content of 0002). Successive memory locations can be displayed by typing CR's. To exit the display mode and return to the command mode type ESC (escape). FANTOM-II will respond with a carriage return and asterisk indicating it is ready for another command. The following example illustrates such a sequence. Characters entered from the keyboard are underlined.

```
♦ M 1 CR
0001 04 CR
0002 7E CR
0003 00 CR
0004 01 CR
0005 36 ESC
♦
```

To change the content of any memory location first display that memory location; then type the replacement value followed by a CR. To return to the command mode type ESC. The following example illustrates such a sequence:

```

♦ M 1      CR
0001 04 3    CR
0002 7E      CR
0003 00      CR
0004 01 0    CR
0005 36      ESC
♦

```

After this sequence of commands memory locations 0001 thru 0005 now contain 03, 7E, 00, 00, and 36, respectively.

V. INSTRUCTION DISPLAY/CHANGE

Type I followed by the address of the first byte of the instruction

The instruction display/change mode is similar to the Memory Display/Change mode except that FANTOM-II prints one instruction per line rather than one memory location per line. An instruction can be one, two, or three bytes, i.e., an OP code or an OP code followed by a one or two byte operand. A CR causes FANTOM-II to sequence to the next instruction. A ESC returns to the command mode. The following example illustrates the instruction display mode:

```

♦ I 0      CR
0000 8D 03 CR
0002 7E 0000 CR
0005 36      CR
0006 37      CR
0007 30      CR
0008 EE 11 CR
000A 39      ESC
♦

```

To change the last byte of a one, two or three byte instruction first display the instruction; then type the desired value of the last byte followed by a CR. (Only the last byte of an instruction can be changed in the instruction mode.) The following example illustrates this mode.

```

♦ I 5 CR
0005 36 CR
0006 37 CR
0007 30 CR
0008 EE 11 0 CR
000A 39 ESC

```

Memory location 0009 now contains 00.

If we type I followed by CR FANTOM-II defaults to the memory location given by the value of the program counter, i.e., FANTOM-II displays the instruction whose first byte is in the memory location given by the value in the program counter register as the following sequence illustrates

```

♦ R C=F0 B=BF A=10 X=FEFF P=0000 S=EDC4
♦ I CR
0000 8D 03 CR
0002 7E 0000 ESC

```

VI. DISPLAY/CHANGE MPU REGISTERS

Simultaneously press CNTL and C, B, A, X, or P.

To display the content of the C, B, A, X, or P register type C, B, A, X, or P while simultaneously depressing CNTL (the control key). FANTOM-II will respond with the register content. To return to the command mode type ESC.

To change the content of the C, B, A, X, or P register first display the content by typing C, B, A, X, or P while simultaneously typing CNTL. After FANTOM-II responds with the register content type the replacement value followed by CR which returns control to the command mode. The following example illustrates register display/change.

```

♦ R C=F0 B=BF A=10 X=0BA4 P=0000 S=EDC4
♦ P=0000 ESC
♦ X=0BA4 0 CR
♦ A=10 FF CR
♦ B=BF FE CR
♦ C=F0 ESC
♦ R C=F0 B=FE A=FF X=0000 P=0000 S=EDC4

```

VII. GO TO USER PROGRAM

Type G followed by carriage return

The G command causes the system to fetch the OP code in the memory location given by the value in the program counter register fetch and start execution. Execution continues until the user program returns control to FANTOM-II or the RESET switch is pushed.

VIII. SINGLE STEP THRU USER PROGRAM

Type S

The S command is similar to the G command except that only the first instruction in the user program is executed and then control is returned to FANTOM-II which responds by executing the R command and then returning to the command mode. That is, the S command causes FANTOM-II to fetch the OP code in the memory location pointed to by the program counter, execute that instruction, then display the contents of the MPU registers, then return to the command mode. The S command is illustrated by the following sequence:

```

♦ R C=F0 B=FE A=FF X=0000 P=0000 S=EDC4
♦ S C=F0 B=FE A=FF X=0000 P=0005 S=EDC2
♦ S C=F0 B=FE A=FF X=0000 P=0006 S=EDC1
♦ S C=F0 B=FE A=FF X=0000 P=0007 S=EDC0
♦ I CR
0007 30 ESC
♦ S C=F0 B=FE A=FF X=EDC1 P=0008 S=EDC0
♦

```

Here the user first checked the MPU registers using the R command. Then the user executed the first instruction by entering S, then executed the second instruction by entering S again, and then executed the third instruction by entering still another S. At this point the user decided to look at the next instruction by entering I followed by a carriage return. Next, he entered ESC to return to the command mode and then S which caused the instruction to be executed.

IX. GO TO USER PROGRAM WITH BREAKPOINTS

Type G followed by the hexadecimal address of the desired breakpoints (separated by commas) followed by two periods.

Type G followed by the hexadecimal address of the first breakpoint, followed by the hexadecimal address of the second breakpoint, etc., followed by two periods. FANTOM-II responds by fetching the OP code stored in the memory location whose address is given by the value of the program counter thus turning system control over to the user program. The program executes until the first breakpoint is encountered whereupon FANTOM-II regains control, executes the R command, and returns to the command mode as illustrated by the following sequence.

```

♦ R C=F0 B=FE A=FF X=0000 P=0000 S=EDC4
♦ G 2,000A..
C=F8 B=FE A=FF X=FEFF P=000A S=EDC0
♦ I CR
000A 39 ESC
♦

```

Here the user entered breakpoints at memory addresses 0002 and 000A. The user program executed until the program counter pointed to memory address 000A at which time control was returned to FANTOM-II which displayed the MPU registers and returned to the command mode. The user then displayed the instruction in memory location 000A.

Breakpoints can be placed only in RAM and only at addresses containing an OP code.

X. DUMP MEMORY

Type D followed by the starting and ending memory address (separated by a comma) followed by a carriage return.

To display an area of memory type D followed by the address of the first memory location to be displayed, followed by a comma, followed by the address of the last memory location to be displayed, followed by a carriage return. FANTOM-II responds by displaying the memory contents in hexadecimal, separated by spaces, with sixteen bytes per line. Each line starts with the address of the first byte of that line. In the following example the user dumps the 32 bytes stored in memory locations EDC0 thru EDDF:

```

♦ D EDC0,EDDF CR
EDC0 0A FE FF 00 02 25 5B 03 10 FB 00 76 32 53 76 5B
EDD0 10 FF 00 76 06 22 0E 02 FD 03 FD DD FC D4 24 FD
♦

```

XI. PUNCH MEMORY

Type R followed by the starting and ending addresses (separated by a comma) followed by a carriage return.

To punch, print, or display an area of memory in MIKBUGtm format, type P followed by the address of the first memory location to be displayed, followed by a comma, followed by the address of the last memory location to be displayed, followed by a carriage return. FANTOM-II responds by punching the memory contents in the same format as Motorola's MIKBUGtm. The MIKBUGtm format is described in section XIV. In the following example the user punches the first sixteen bytes of memory.

◆ P 0,F CR

S11300008D037E0000363730EE0039020202020210S9

If the beginning and ending addresses are omitted, i.e., if the user types P followed by a carriage return, FANTOM-II uses the addresses specified in the most recent punch or dump command.

When using a manual punch, the punch should be enabled before typing CR.

XII. LOAD MEMORY

Type L

To load memory type L. Then enter the MIKBUGtm formatted data. FANTOM-II responds by ignoring all characters prior to the Start-of-Record character "S1". The data following the "S1" are stored in memory and the checksum verified. If a checksum is in error loading is stopped and a question mark is displayed. The following example illustrates the use of the load command to load from paper tape:

◆ L S11300008D037E0000363730EE0039020202020210 (CR/LF)

S9

The "S9" could have been included on the paper tape instead of being entered from the keyboard. In either case "S9" causes FANTOM-II to return to the command mode.

The MIKBUGtm format is described in section XIV.

XIII. INTERRUPT VECTORS

Software Interrupt. If the processor encounters an SWI instruction FANTOM-II responds by pushing the MPU registers onto the users stack and then vectors to memory address EDFA. Memory addresses EDFA, EDFB, and EDFC are reserved for the users SWI handler, or a jump instruction to the SWI handler at another memory location.

Interrupt Request. If the IRQ line is pulled low and the interrupt mask bit is not set (bit 1 in the CCR is clear) FANTOM-II responds by pushing the MPU registers onto the user stack and then vectors to memory address EDF7. Memory locations EDF7, EDF8, and EDF9, are reserved for the users IRQ handler, or a jump instruction to the IRQ handler at another memory location.

Non Maskable Interrupt. If the NMI line is pulled low FANTOM-II responds by pushing the MPU registers onto the user stack and then vectoring to memory address EDFD. Memory locations EDFD, EDFF, and EDFF are reserved for the user's NMI handler, or a jump instruction to another memory address.

XIV. INPUT FORMAT

Whenever FANTOM-II is expecting a hexadecimal input, the user is free to delete leading zeros, use spaces, and repeat incorrect entries. For example, if FANTOM-II is expecting a 4 digit hexadecimal number any of the following entries will result in FANTOM-II accepting 0396:

```
0396 CR
0 3 9 6 CR
396 CR
397 0396 CR
```

FANTOM-II assumes leading zeros if the entry is less than four hexadecimal characters, ignores all imbedded spaces, and accepts only the last four characters before the carriage return.

XV. MIKBUGtm FORMAT

The MIKBUGtm format is a character file, each line of which is a record. A record has the following format:

<header> <byte count> <address> <data> <checksum>

<Header> tells what type of information is contained in the record: "S1" indicates data to be loaded into memory; "S9" means that no more records follow. All characters following "S9" are ignored.

<byte count> is a one-byte hexadecimal number that tells how many bytes follow, including the address, data, and checksum.

<address> is a two-byte hexadecimal number that gives the starting memory address.

<data> is the bytes to be loaded into memory or punched from memory. All data is represented in hexadecimal, two characters per byte.

<checksum> is a byte-wise checksum of everything except the header.

WINTEK CORP.
902 N. 9 St.
LAFAYETTE, IN 47904
317-742-6802

FANTOM II RAM DECLARATIONS.

EDBE		ORG	\$EDBE	
EDBD	USRSTK	EQU	♦-1	INITIAL VALUE OF USER S.P.
EDBE	USRCCR	RMB	1	INITIAL LOCATIONS OF USER
EDBF	USRACB	RMB	1	REGISTERS - VALID ON POWER
EDC0	USRACA	RMB	1	UP ONLY!!! USER MAY
EDC1	USRIND	RMB	2	CHANGE SUBSEQUENTLY
EDC3	USRPCT	RMB	2	
EDC5		RMB	37	MONITOR STACK AREA
EDE9	MONSTK	EQU	♦-1	INITIAL MONITOR S.P.
EDEA	END	RMB	2	PUNCH END ADDRESS
EDEC	BEG	RMB	2	PUNCH BEG ADDRESS
EDEE	TEMP	RMB	2	TEMP WORK AREA
EDF0	USERS	RMB	2	USER STACK POINTER SAVE
EDF2	TEM1	RMB	2	MONITOR STACK POINT STORE
EDF4	SYSSWI	RMB	3	SYSTEM SWI VECTOR
EDF7	UIRQ	RMB	3	USER IRQ VECTOR
EDFA	USWI	RMB	3	USER SWI VECTOR
EDFD	UNMI	RMB	3	USER NMI VECTOR

♦♦ ACIA ADDRESSES.

EE08	ACIACR	EQU	\$EE08
EE08	ACIASR	EQU	\$EE08
EE09	ACIADR	EQU	\$EE09

♦♦ FANTOM COMMAND HANDLER RULES:

♦ ALL FANTOM COMMAND HANDLERS MUST HAVE THEIR
♦ ENTRY POINTS INSIDE THE SAME 256 BYTE PAGE
♦ OF ROM SO THAT THEIR ENTRY POINT ADDRESS
♦ MAY BE STORED IN A SINGLE BYTE.

♦ ALL HANDLERS ARE CALLED WITH AN EFFECTIVE
♦ -JSR- AND HAVE THE FOLLOWING ADDITIONAL
♦ REQUIREMENTS:

♦ ENTRY: (B) = 0
♦ (X) = (USER STACK POINTER)
♦ (A) = ASCII COMMAND CHARACTER

FC00		ORG	\$FC00
FC00	ROMBAS	EQU	♦

♦♦ GO--GO TO USER PROGRAM

♦
♦ ACCEPTS HEX ADDRESSES, INSERTS BREAKPOINTS,
♦ SETS UP SWI RETURN AND
♦ BEGINS EXECUTION OF USER CODE
♦

♦ ENTRY--(B) = 0

♦ ROM (RETURN) ADDRESS ONTO STACK

♦ WE WILL TRY TO CLEAR THIS LOCATION
♦♦ EXIT--REMOVES BREAKPOINTS AND RETURNS TO
♦ THE MAIN MONITOR LOOP
♦

♦ USES--ALL REGISTERS

FC00	CE ED FA	GO	LDX	#USWI	INCHASE NO BREAKPOINTS
FC03	20 0E		BRA	GO2	
FC05	36	GO1	PSH A		ADDRESS INTO STACK
FC06	37		PSH B		
FC07	30		TSX		
FC08	EE 00		LDX	0,X	
FC0A	E6 00		LDA B	0,X	
FC0C	86 3F		LDA A	#3F	REPLACE OPCODE WITH SWI
FC0E	A7 00		STA A	0,X	
FC10	CE FE 4F		LDX	#605	BREAKPOINTS ARE ALIVE
FC13	37	GO2	PSH B		PUSH OPCODE ONTO STACK
FC14	8D 4E		BSR	AHV	
FC16	25 ED		BCS	GO1	
FC18	7E FE 49		JMP	GO4	GIVES CONTROL TO USER CODE

♦♦ PUNCH--PROCESS PUNCH COMMAND

♦ ENTRY--DEFAULT VALUES ON STACK BELOW RET. ADDRESS

♦ RETURN ADDRESS
♦ RETURN ADDRESS
♦ END ADDRESS
♦ END ADDRESS (LO)
♦ BEG ADDRESS
♦ BEG ADDRESS (LO)
♦♦ EXIT--AFTER ACCEPTING USER PECIFIED BEGINNING
♦ AND ENDING ADDRESSES, THIS SUBROUTINE
♦ PUNCHES OR DUMPS, AND THEN RETURNS
♦ TO THE CALLING ROUTINE. IT IS
♦ CALLERS RESPONSIBILITY TO
♦ CLEAN UP STACK

FC1B	4F	PUNCH	CLR A	DUMP WITH SPACES
FC1C	5C	PNCH1	INC B	NORMAL DUMP
FC1D	36		PSH A	
FC1E	37		PSH B	
FC1F	30		TSX	

PUNCH - PROCESS ♦PUNCH♦ COMMAND.

FC20	8D 42		BSR	AHV	GET BEGINNING ADDRESS
FC22	24 0A		BCC	PNCH2	USE DEFAULT INSTEAD
FC24	A7 07		STA A	7,X	STORE BEGINNING ADDRESS
FC26	E7 06		STA B	6,X	
FC28	8D 3A		BSR	AHV	END ADDRESS
FC2A	A7 05		STA A	5,X	
FC2C	E7 04		STA B	4,X	
FC2E	32	PNCH2	PUL A		NOW SET UP ACIA
FC2F	6D 01		TST	1,X	
FC31	27 08		BEQ	PNCH3	
FC33	B7 EE 08 ✓		STA A	ACIACR	
FC36	86 12		LDA A	#\$12	TAPE ON
FC38	BD FD 80		JSR	OUTCH	
FC3B	A6 07	PNCH3	LDA A	7,X	
FC3D	36		PSH A		
FC3E	A6 06		LDA A	6,X	CURRENT ADDRESS ONTO STACK
FC40	36		PSH A		
FC41	7E FD E9		JMP	PNCH4	

♦♦ MEM - DISPLAY MEMORY.

♦

♦

♦

♦

♦

♦

ENTRY: (B) = 0

(X) = USER STACK POINTER

EXIT: BYTES TYPED/REPLACED UNTIL 'ESC'

VIA RTS TO MAIN MONITOR LOOP

FC44	5A	MEM	DEC B	BYTE DISPLAY
FC45	37	INST	PSH B	INSTRUCTION DISPLAY
FC46	EE 06		LDX	6,X
FC48	8D 1A		BSR	AHV
FC4A	24 07		BCC	MEM1
FC4C	36		PSH A	
FC4D	37		PSH B	
FC4E	30		TSX	
FC4F	EE 00		LDX	0,X
FC51	31		INS	
FC52	31		INS	
FC53	0C	MEM1	CLC	INSURES WE BRANCH
FC54	33	MEM2	PUL B	RETRIEVE MODE FLAG
FC55	24 03		BCC	MEM3
FC57	8D 43		BSR	STORE0
FC59	08		INX	
FC5A	BD FD 5B	MEM3	JSR	TYPINS
FC5D	37		PSH B	
FC5E	8D 04		BSR	AHV
FC60	23 F2		BLS	MEM2
FC62	33	MEM4	PUL B	
FC63	39		RTS	

TYPE THE DATA

GET REPLACEMENT VALUE
IF CR OR VALID ENTRY

♦♦ AHV - ACCUMULATE HEX VALUE.
♦
♦ READS HEX DIGITS FROM THE KEYBOARD; ACCUMULATES
♦ THEM INTO A 16 BIT VALUE.
♦
♦ ENTRY: NONE
♦
♦ EXIT (BA) = ACCUMULATED VALUE IF VALID ENTRY
♦ (A) = ASCII VALUE OTHERWISE
♦ 'C' CLEAR IF NO VALID ENTRY
♦ 'Z' SET IF LAST CHARACTER TYPED WAS 'CR'.
♦
♦ USES: A,B,C

FC64	5F	AHV	CLR B	
FC65	8D 6B		BSR	IHD GET FIRST DIGIT
FC67	24 1C		BCC	AHV3
FC69	36	AHV1	PSH A	
FC6A	37		PSH B	
FC6B	48		ASL A	
FC6C	59		ROL B	
FC6D	48		ASL A	
FC6E	59		ROL B	
FC6F	48		ASL A	
FC70	59		ROL B	
FC71	48		ASL A	
FC72	59		ROL B	
FC73	37		PSH B	
FC74	36		PSH A	MAKE WAY FOR NEXT DIGIT
FC75	8D 5B		BSR	IHD
FC77	24 07		BCC	AHV2 THAT'S ALL FOLKS
FC79	33		PUL B	
FC7A	1B		ABA	
FC7B	33		PUL B	
FC7C	31		INS	SKIP PREVIOUS VALUE
FC7D	31		INS	
FC7E	20 E9		BRA	AHV1
FC80	31	AHV2	INS	SKIP LATEST VALUE
FC81	31		INS	
FC82	33		PUL B	
FC83	32		PUL A	GET OLD VALUE
FC84	0D		SEC	
FC85	39	AHV3	RTS	

♦♦ REG - REGISTER DISPLAY COMMANDS.
♦
♦ ENTRY: (X) = USER STACK POINTER
♦ (B) = 0
♦
♦ EXIT: REGISTER CHANGED IF VALID HEX
♦ UNCHANGED OTHERWISE

FC86	08	REGP	INX
FC87	08		INX

REG - REGISTER DISPLAY COMMANDS.

FC88	08	REGX	INX		
FC89	5C		INC B		
FC8A	08	REGA	INX		
FC8B	08	REGB	INX		
FC8C	8B 40	REGC	ADD A	#340	
FC8E	8D 37		BSR	PTREGA	
FC90	37		PSH B		
FC91	8D D1		BSR	AHV	GET REPLACEMENT VALUE
FC93	24 CD		BCC	MEM4	NO REPLACE - PUL B, RTS
FC95	8D 05		BSR	STORE0	
FC97	17		TBA		
FC98	33		PUL B		
FC99	5A		DEC B		
FC9A	27 07		BEQ	STORE1	

♦♦ STORE - STORE AND CHECK BYTE
 ♦
 ♦ ENTRY: (X) = ADDRESS TO STORE
 ♦ EXIT: (A) STORED AT ADDRESS
 ♦ ERROR RETURNS TO MAIN MONITOR LOOP
 ♦
 ♦ USES: C

FC9C	09	STORE0	DEX		
FC9D	A7 00	STORE	STA A	0,X	
FC9F	A1 00		CMP A	0,X	
FCA1	26 4E		BNE	LOAD7	
FCA3	39	STORE1	RTS		

♦♦ EXEC - PROCESS 'EXEC' COMMAND.
 ♦
 ♦ HERE WE CALL 'SSTEP' TO EXECUTE ONE INSTRUCTION
 ♦ FROM THE USER CODE AND THEN RETURN TO PRINT
 ♦ HIS REGISTERS

FCA4	BD FE C1	EXEC	JSR	SSTEP	
------	----------	------	-----	-------	--

♦♦♦ REGS - DISPLAY ALL REGISTERS.
 ♦
 ♦ PRINTS REGISTERS IN THE FORM:
 ♦
 ♦ 'C=___ B=___ A=___ X=_____ P=_____ S=_____'

FCA7	BD FE 8A	REGS	JSR	CLBLDX	
FCA8	86 43		LDA A	#'C'	
FCA9	8D 19		BSR	PTREGA	
FCAE	86 42		LDA A	#'B'	
FCB0	8D 17		BSR	PTREG	
FCB2	86 41		LDA A	#'A'	
FCB4	8D 13		BSR	PTREG	
FCB6	86 58		LDA A	#'X'	
FCB8	8D 0E		BSR	PTREGB	

FCBA	86 50	LDA A	#'P'
FCBC	8D 0B	BSR	PTREG
FCBE	86 53	LDA A	#'S'
FCC0	09	DEX	
FCC1	FF ED EE	STX	TEMP
FCC4	CE ED ED	LDX	#TEMP-1

♦♦ OUTPUT ROUTINES

♦
♦ ENTRY: (A) = REGISTER NAME
♦ (X) = ADDRESS OF STORED REG
♦ (B) = NUMBER BYTES TO PRINT
♦ EXIT: REGISTER PRINTED
♦ (X) = ADDRESS OF NEXT REGISTER
♦ USES: A,X

FCC7	08	PTREGA	INX	SKIP A BYTE FIRST.
FCC8	5C	PTREGB	INC B	PRINT MORE BYTES
FCC9	8D 78	PTREG	BSR	OUTCHA
FCCB	86 3D		LDA A	#'='
FCCD	8D 74		BSR	OUTCHA
FCCF	7E FD 74		JMP	OUTREG

ENTRY CONDITIONS APPLY HERE

♦♦ IHD - INPUT HEX DIGIT.

♦
♦ INPUTS A DIGIT FROM THE CONSOLE; CHECKS TO SEE
♦ IF IT IS A VALID HEX DIGIT.

♦ ENTRY: NONE
♦
♦ EXIT: (A) = HEX VALUE IF VALID ENTRY
♦ = ASCII VALUE OTHERWISE
♦ 'C' SET IF HEX
♦ 'Z' SET IF CR

FCD2	8D 62	IHD	BSR	INCH	
FCD4	81 20		CMP A	#' '	
FCD6	27 FA		BEQ	IHD	
FCD8	80 30		SUB A	#'0'	
FCD9	25 0C		BCS	IHD1	LESS THAN ZERO
FCDC	81 0A		CMP A	#10	
FCDE	25 7A		BCS	IHD3	ZERO THRU NINE
FCE0	80 11		SUB A	#'A'-'0'	DISPLACE A TO ZERO
FCE2	81 06		CMP A	#6	
FCE4	25 72		BCS	IHD2	A THRU F
FCE6	8B 11		ADD A	#'A'-'0'	
FCE8	8B 30	IHD1	ADD A	#'0'	DISPLACE BACK
FCEA	81 0D		CMP A	#8D	SET 'Z' IF CR
FCEC	0C		CLC		
FCED	39		RTS		

♦♦ LOAD - PROCESS *LOAD* COMMAND.
 ♦
 ♦ LOADS MEMORY FROM CASSETTE OR PAPER TAPE
 ♦ IN THE MIKBUG(TM) FORMAT
 ♦
 ♦ ENTRY: (B) = 0
 ♦ EXIT: VIA RTS EXCEPT ON ERROR - THEN TO
 ♦ MAIN MONITOR LOOP.
 ♦
 ♦ USES: ALL
 ♦
 ♦ ALL DONE - CHECK CHECKSUM

FCEE 4C LOAD5 INC A
 FCEF 27 15 BEQ LOAD3

♦ ERROR RETURN JUMPS HERE

FCF1 BD FD A6 LOAD7 JSR CRLF
 FCF4 86 3F LDA A #'?'
 FCF6 7E FE 2E JMP MAIN1

♦ NORMAL ENTRY IS AT 'LOAD'
 ♦ OPTIONAL CASSETTE ENTRY IS 'LOAD1'

FCF9 5A	LOAD1	DEC B	CASSETTE ENTRY
FCFA F7 EE 08		STA B ACIACR	RESET ACIA
FCFD CB 09	LOAD	ADD B #\$9	NORMAL ENTRY
FCFF F7 EE 08		STA B ACIACR	FIX ACIA
FD02 86 11		LDA A #\$11	READER-ON
FD04 8D 3D		BSR OUTCHA	
FD06 8D 2E	LOAD3	BSR INCH	
FD08 81 53		CMP A #'S'	
FD0A 26 FA		BNE LOAD3	
FD0C 8D 28		BSR INCH	
FD0E 80 39		SUB A #'9'	
FD10 27 48		BEQ IHD3	RETURN TO CALLER
FD12 80 F8		SUB A #'1'-'9'	CLEAR A IF S1
FD14 26 F0		BNE LOAD3	

♦ WE HAVE 'S1' -- A WILL BE CHECKSUM

FD16 8D 2D	BSR IHB	GET BYTE COUNT
FD18 C0 02	SUB B #2	
FD1A F7 ED F2	STA B TEM1	
FD1D 8D 26	BSR IHB	GET ADDRESS
FD1F 37	PSH B	
FD20 37	PSH B	
FD21 8D 22	BSR IHB	
FD23 30	TSX	
FD24 E7 01	STA B 1,X	
FD26 EE 00	LDX 0,X	
FD28 31	INS	
FD29 31	INS	PLACE INTO X REG
FD2A 8D 19	BSR IHB	GET NEXT BYTE
FD2C 7A ED F2	DEC TEM1	
FD2F 27 BD	BEQ LOAD5	

FD31	E7 00	STA B	0,X
FD33	08	INX	
FD34	20 F4	BRA	LOAD4

♦♦ INCH - INPUT CHARACTER FROM CONSOLE
♦
♦ ENTRY: NONE
♦
♦ EXIT: (A) = ASCII VALUE; RUBOUTS MASKED
♦
♦ USES A,C

FD36	B6 EE 08	INCH	LDA A ^{JMP 2000} ACIADR	GET RX STATUS
FD39	44		LSR A	ROTATE RX BIT INTO C
FD3A	24 FA	01's	BCC INCH	
FD3C	B6 EE 09		LDA A ACIADR	GET DATA
FD3F	81 7F		CMP A #\$7F	IGNORE RUBOUTS
FD41	27 F3		BEQ INCH	
FD43	20 3B	OUTCHA	BRA OUTCH	SHORT JUMP EXTENSION

♦♦ IHB - INPUT HEX BYTES.
♦
♦ ENTRY: (A) = CHECKSUM
♦
♦ EXIT: (A) = CHECKSUM + NEW DIGIT
♦ (B) = NEW DIGIT

FD45	36	IHB	PSH A	CHECKSUM ONTO STACK
FD46	8D 8A		BSR IHD	GET FIRST DIGIT
FD48	24 A7		BCC LOAD7	ERROR RETURN
FD4A	48		ASL A	
FD4B	48		ASL A	
FD4C	48		ASL A	
FD4D	48		ASL A	
FD4E	16		TAB	
FD4F	8D 81		BSR IHD	GET SECOND DIGIT
FD51	24 9E		BCC LOAD7	
FD53	1B		ABA	COMBINE TWO DIGITS
FD54	16		TAB	
FD55	32		PUL A	GET CHECKSUM
FD56	1B		ABA	UPDATE CHECKSUM
FD57	39		RTS	

♦ INPUT HEX DIGIT CONTINUED

FD58	80 F6	IHD2	SUB A #-10	
FD5A	39	IHD3	RTS	HEX RETURN

♦♦ TYPINS - TYPE ONE INSTRUCTION OR BYTE
♦
♦ ENTRY: (X) = FWA OF INSTRUCTION OR BYTE
♦ (B) < 0 THEN BYTE
♦ > 0 THEN INSTRUCTION
♦ EXIT: BYTE/INSTRUCTION TYPED ON NEW LINE
♦ (X) = ADDRESS OF NEXT BYTE/INSTRUCTION
♦ USES: ALL, 'TEMP'

FD5B	A6 00	TYPINS	LDA A	0,X	GET MEMORY BYTE
FD5D	36		PSH A		OPCODE ONTO STACK
FD5E	8D 46		BSR	CRLF	
FD60	CE ED EE		LDX	#TEMP	
FD63	8D 15		BSR	OUT4HS	
FD65	32		PUL A		
FD66	5D		TST B		IS THIS INSTRUCTION MODE?
FD67	2B 06		BMI	TYPIN1	NO IT ISN'T
FD69	BD FE 8F		JSR	BYTCNT	
FD6C	5A		DEC B		
FD6D	2B 82		BMI	LOAD7	NOT AN INSTRUCTION
FD6F	FE ED EE	TYPIN1	LDX	TEMP	
FD72	8D 08		BSR	OUT2HS	

♦♦ OUTREG - OUTPUT BYTE OR REGISTER
♦
♦ ENTRY: (X) = ADDRESS OF BYTE OR STORED REGISTER
♦ (B) < 1 THEN NO BYTES TYPED
♦ = 1 THEN ONE BYTE TYPED
♦ > 2 THEN TWO BYTES TYPED
♦
♦ EXIT: X INCREMENTED PAST BYTE(S)
♦ (A) = ' '
♦ USES: A,B,C,X

FD74	C1 01	OUTREG	CMP B	#1	HOW MANY MORE BYTES?
FD76	2B 14		BMI	OUT2	
FD78	27 02		BEQ	OUT2HS	

♦♦ OUT4HS - OUTPUT FOUR HEX DIGITS AND SPACE
♦
♦ ENTRY: (X) = ADDRESS OF BYTES
♦ EXIT: X INCREMENTED PAST BYTES
♦ (A) = ' '
♦ USES: A,X

FD7A	8D 13	OUT4HS	BSR	THB	
------	-------	--------	-----	-----	--

♦♦ OUT2HS - OUTPUT TWO HEX DIGITS AND SPACE
♦ ENTRY,EXIT,USES - SEE 'OUT4HS'

FD7C	8D 11	OUT2HS	BSR	THB	
------	-------	--------	-----	-----	--

FANTOM II OUTPUT ROUTINES

♦♦ OUTSP - OUTPUT SPACE CODE
♦ ENTRY: NONE; EXIT: (A) = SPACE; USES: A,C

FD7E 86 20 OUTSP LDA A #'

♦♦ OUTCH - OUTPUT CHARACTER TO CONSOLE
♦ ENTRY: (A) = ASCII VALUE OF CHARACTER
♦ EXIT: (A) = ASCII VALUE

♦ LABEL OPERATION USES: CCR EXCEPT 'C' BIT

FD80	37	OUTCH	PSH B	JMP 2C03	
FD81	C6 02		LDA B #2		MASK FOR ACIACR
FD83	F5 EE 08	OUT1	BIT B	ACIASR	
FD86	27 FB	01's	BEQ	OUT1	
FD88	B7 EE 09		STA A	ACIADR	SEND (A) TO CONSOLE
FD8B	33		PUL B		
FD8C	39	OUT2	RTS		Comments

♦♦ PUNT2 - PUNCH TWO DIGITS AND UPDATE CHECKSUM
♦ ENTRY: (B) = CHECKSUM
♦ (X) = ADDRESS
♦ EXIT: X INCREMENTED; CHECKSUM UPDATED
♦ USES: A,B,C,X

FD8D EB 00 PUNT2 ADD B 0,X UPDATE CHECKSUM

♦♦ THB - TYPE HEX BYTES.
♦ ENTRY: (X) = ADDRESS OF BYTE TO OUTPUT IN HEX
♦ EXIT: (X) = OLDVALUE+1
♦ USES: A,X,C

FD8F A6 00 THB LDA A 0,X
FD91 08 INX

♦♦ THB0 - TYPE (A) IN HEX
♦ ENTRY: (A) = VALUE
♦ EXIT: VALUE TYPED
♦ USES: A,C

FD92	36	THB0	PSH A	0010 1111
FD93	44		LSR A	0000 0010
FD94	44		LSR A	
FD95	44		LSR A	
FD96	44		LSR A	
FD97	8D 03		BSR	THB1
FD99	32		PUL A	

0000 0010
0000 1010
1111
0111
1010

FD9A	84	0F		AND A	#3F
FD9C	81	0A	THB1	CMP A	#10
FD9E	25	02		BCS	THB2
FDA0	8B	07		ADD A	#7
FDA2	8B	30	THB2	ADD A	#30
FDA4	20	DA	OUTCHB	BRA	OUTCH

SHORT JUMP EXTENSION

♦♦ CRLF - TYPE CARRIAGE RETURN/LINEFEED
♦
♦ ENTRY - NONE
♦
♦ EXIT - X POINTS PAST CR/LF STRING TO 'S1' STRING
♦ IF ENTERED AT CRLF, X STORED IN 'TEMP'

FDA6	FF	ED	EE	CRLF	STX	TEMP
FDA9	CE	FF	B4	CRLF1	LDX	#CRSTR
FDA0	20	51			BRA	OUTSTR

♦ REST OF PUNCH ROUTINE

FDAE	C1	0F	PNCH6	CMP B	#15	
FDB0	25	02		BCS	PNCH8	
FDB2	C6	0F	PNCH7	LDA B	#15	
FDB4	37		PNCH8	PSH B		BYTE COUNT ON STACK
FDB5	E6	02		LDA B	2,X	GET PUNCH FLAG
FDB7	8D	F0		BSR	CRLF1	
FDB9	5D			TST B		PUNCH S1, BYTECOUNT?
FDBA	27	09		BEQ	PNCH9	NO
FDBC	8D	41		BSR	OUTSTR	
FDBE	33			PUL B		
FDBF	37			PSH B		GET BYTE COUNT
FDC0	CB	04		ADD B	#4	FRAME COUNT
FDC2	17			TBA		
FDC3	8D	CD		BSR	THB0	
FDC5	30		PNCH9	TSX		
FDC6	08			INX		POINT AT ADDRESS WORD
FDC7	8D	C4		BSR	PUNT2	
FDC9	8D	C2		BSR	PUNT2	
FDCB	30		PNCH85	TSX		
FDC0	6D	03		TST	3,X	SPACES REQUIRED?
FDC2	26	02		BNE	PNCH10	
FDD0	8D	AC		BSR	OUTSP	
FDD2	EE	01	PNCH10	LDX	1,X	
FDD4	8D	B7		BSR	PUNT2	
FDD6	30			TSX		
FDD7	6C	02		INC	2,X	INCREMENT ADDRESS POINTER ON
STACK						
FDD9	26	02		BNE	PNCH11	
FDDB	6C	01		INC	1,X	
FDDD	6A	00	PNCH11	DEC	0,X	
FDDF	2A	EA		BPL	PNCH85	
FDE1	32			PUL A		\$FF INTO A
FDE2	6D	03		TST	3,X	CHECKSUM?

FDE4	27 03		BEQ	PNCH4	
FDE6	10		SBA		B-BAR INTO A
FDE7	8D A9		BSR	THB0	
FDE9	30	PNCH4	TSX		
FDEA	E6 06		LDA B	6,X	END INTO (AB)
FDEC	A6 05		LDA A	5,X	
FDEE	E0 01		SUB B	1,X	END - CURRENT INTO (AB)
FDF0	A2 00		SBC A	0,X	
FDF2	22 BE		BHI	PNCH7	MORE THAN 256
FDF4	24 B8		BCC	PNCH6	MORE THAN ZERO
FDF6	32		PUL A		
FDF7	32		PUL A		
FDF8	32		PUL A		
FDF9	4D		TST A		
FDFB	27 0A		BEQ	OUTST3	
FDFC	CE FF B2		LDX	#S9MSG	

♦♦ OUTSTR -- OUTPUT CHARACTER STRING TO CONSOLE
♦
♦ ENTRY -- (X) = ADDRESS OF STRING
♦
♦ EXIT -- (X) INCREMENTED PAST END OF STRING
♦ END OF STRING MARKER IS NEGATIVE
♦ USES -- A,X,C

FDFE	A6 00	OUTSTR	LDA A	0,X
FE01	08		INX	
FE02	8D A0	OUTST2	BSR	OUTCHB
FE04	2A F9		BPL	OUTSTR
FE06	39	OUTST3	RTS	

♦♦ RESET -- POWER ON/USER CRASH RESET ROUTINE
♦
♦ SET UP ACIA, CHECK IF BREAKPOINTS ALIVE
♦ AND ATTEMPT TO CLEAR THEM.

FE07	CE EE 07	RESET	LDX	#ACIACR-1	
FE0A	EF 00		STX	0,X	RESET ACIA
FE0C	09	RESET1	DEX		RESET DELAY
FE0D	26 FD		BNE	RESET1	

♦ NOW TRY TO CLEAR ALL BREAKPOINTS

FE0F	8E ED BD		LDS	#USRSTK	
FE12	BF ED F0		STS	USERS	INITIALIZE FOR USER
FE15	BE ED F2		LDS	TEM1	
FE18	FE ED F5	RESET2	LDX	SYSSWI+1	SEE IF BREAKPOINTS ACTIVE
FE1B	7F ED F5		CLR	SYSSWI+1	
FE1E	8C FE 4F		CPX	#G05	
FE21	26 0A		BNE	MAINLP	
FE23	32	RESET3	PUL A		
FE24	30		TSX		

FE25	EE 00	LDX	0,X	
FE27	31	INS		
FE28	31	INS		
FE29	A7 00	STA A	0,X	LAST ONE IS ZERO
FE2B	26 F6	BNE	RESET3	BUT IT'S PLACED INTO ROM

♦♦ MAIN MONITOR LOOP - SENDS PROMPT AND ACCEPTS
♦ COMMANDS; ENTRY 'MAIN1' IS ERROR RETURN

FE2D	4F	MAINLP	CLR A	(A) SENT BEFORE PROMPT
FE2E	8E ED E9	MAIN1	LDS #MONSTK	ENTER HERE FOR ERROR
FE31	C6 41		LDA B #B41	7+E+2,1/16,RTS FALSE
FE33	F7 EE 08		STA B ACIACR	
FE36	CE FF A9		LDX #PMTMSG	
FE39	8D 37		BSR CMDEX	
FE3B	20 F0		BRA MAINLP	

♦♦ BREAKPOINT RETURN

♦
♦ 'GO' COMMAND HANDLER HAS SET UP SYSTEM
♦ SWIVECTOR TO HERE IF THERE ARE ANY
♦ BREAKPOINTS. WE MUST EXAMINE BREAKPOINT
♦ TABLE TO SEE IF SWI IS BREAKPOINT OR
♦ USER CODE. THE SWI MUST BE INTERPRETED
♦ IF IT IS PART OF USER CODE

FE3D	08	603	INX	
FE3E	08		INX	
FE3F	08		INX	
FE40	6D 00		TST 0,X	LOOK FOR LAST BREAKPOINT
FE42	26 1E		BNE 607	
FE44	8D 7B		BSR SSTEP	STEP PAST USER SWI
FE46	CE FE 4F		LDX #605	
FE49	BF ED F2	604	STS TEM1	
FE4C	7E FF 24		JMP SWIVE1	
FE4F	30	605	TSX	RETURN HERE ON BREAKPOINT
FE50	BF ED F0		STS USERS	
FE53	BE ED F2		LDS TEM1	
FE56	A6 06		LDA A 6,X	DECREMENT USER PC
FE58	26 02		BNE 606	AND SAVE A COPY IN (BA)
FE5A	6A 05		DEC 5,X	
FE5C	E6 05	606	LDA B 5,X	
FE5E	4A		DEC A	
FE5F	A7 06		STA A 6,X	
FE61	30		TSX	
FE62	E1 01	607	CMP B 1,X	SEARCH STACK FOR BKPOINT
FE64	26 D7		BNE 603	
FE66	A1 02		CMP A 2,X	
FE68	26 D3		BNE 603	
FE6A	BD FD A6		JSR CRLF	
FE6D	BD FC A7		JSR REGS	PRINT USER REGISTERS

FE70 20 A6

BRA RESET2

♦♦ CMDEX - COMMAND EXECUTE SUBROUTINE
♦
♦ COMPUTES AN ADDRESS BASED UPON COMMAND INPUT
♦ AND JUMPS TO CORRESPONDING HANDLER
♦
♦ ENTRY: RETURN ADDRESS ON STACK
♦
♦ EXIT: TO COMMAND HANDLER
♦ (B) = 0
♦ (X) = USER STACK POINTER
♦
♦ USES: ALL,TEMP,TEM1

FE72	8D 8E	CMDEX	BSR	OUTST2	SEND PROMPT
FE74	BD FD 36		JSR	INCH	GET COMMAND
FE77	36		PSH A	0C	
FE78	4A		DEC A	0B	
FE79	08	CMDE1	INX		
FE7A	A1 09		CMP A	CMDDSP,X	
FE7C	25 FB		BCS	CMDE1	
FE7E	26 AD		BNE	MAINLP	ILLEGAL COMMAND
FE80	BD FD 7E		JSR	OUTSP	
FE83	32		PUL A		
FE84	E6 17		LDA B	CMDTEL+CMDDSP,X	
FE86	37		PSH B		
FE87	06 FC		LDA B	#ROMBAS/256	
FE89	37		PSH B		HANDLER ADDRESS ON STACK

♦♦ CLBLDX - SET B = 0; X = USERS

FE8A 5F CLBLDX CLR B

♦♦ LDXU - SET (X) = USER S.P.

FE8B	FE ED F0	LDXU	LDX	USERS	
FE8E	39		RTS		EXIT TO COMMAND HANDLER

♦♦ BYTCNT - DETERMINE NUMBER OF BYTES IN AN INSTRUCTION

♦
♦ ENTRY--OPCODE IN A

♦ EXIT--(B) = NUMBER OF BYTES IN INSTRUCTION
♦ (I.E. 0,1,2, OR 3)

♦ 'C' CLEAR IF RELATIVE ADDRESSING
♦ 'Z' SET IF ILLEGAL INSTRUCTION

♦ USES--A,B,C,X

FE8F	36	BYTCNT	PSH A	
FE90	16		TAB	
FE91	CE FF D7		LDX	#OPTAB-1
FE94	08	BYT1	INX	
FE95	C0 08		SUB B	#8
FE97	24 FB		BCC	BYT1
FE99	A6 00		LDA A	0,X
FE9B	46	BYT2	ROR A	
FE9C	5C		INC B	
FE9D	26 FC		BNE	BYT2
FE9F	32		PUL A	
FEA0	25 1E		BCS	BYT7
FEA2	81 30		CMP A	#\$30
FEA4	24 04		BCC	BYT3
FEA6	81 20		CMP A	#\$20
FEA8	24 14		BCC	BYT5
FEAA	81 60	BYT3	CMP A	#\$60
FEAC	25 11		BCS	BYT6
FEAE	81 8D		CMP A	#\$8D
FEB0	27 0C		BEQ	BYT5
FEB2	84 BD		AND A	#\$BD
FEB4	81 8C		CMP A	#\$8C
FEB6	27 04		BEQ	BYT4
FEB8	84 30		AND A	#\$30
FEBA	81 30		CMP A	#\$30
FEBC	C2 FF	BYT4	SBC B	#\$FF
FEBE	5C	BYT5	INC B	
FEBF	5C	BYT6	INC B	
FEC0	39	BYT7	RTS	

CHECK FOR BRANCH

IT IS A BRANCH

IT IS ONE BYTE

IT IS BSR

IS X OR SP IMMEDIATE
CHECK FOR THREE BYTES

♦♦ SSTEP - PERFORM SINGLE STEP.

♦ THIS SUBROUTINE MAY BE CALLED TO EXECUTE A
♦ SINGLE INSTRUCTION FROM THE USER CODE. ILLEGAL
♦ INSTRUCTIONS ARE CHECKED AND THE STEP ROUTINE
♦ REFUSES TO EXECUTE THEM.

♦ MOST INSTRUCTIONS ARE EXECUTED OUT-OF-PLACE;
♦ SOME INSTRUCTIONS (THOSE WHICH INVOLVE THE
♦ P.C.) MUST BE INTERPRETED. A SET OF HANDLERS
♦ IS INCLUDED FOR THIS SET OF INSTRUCTIONS

FEC1 BF ED EE SSTEP STS TEMP WE'LL USE THIS TO RETURN

FEC4	8D 05		BSR	LDXU	
FEC6	A6 07		LDA A	7,X	PUSH USER PC ONTO STACK
FEC8	36		PSH A		
FEC9	A6 06		LDA A	6,X	
FECB	36		PSH A		
FECC	EE 06		LDX	6,X	NOW GET USER PC INTO X
FECE	86 3F		LDA A	#\$3F	SWI'S ARE NORMAL EXIT FROM
FED0	36		PSH A		SCRATCH PAD EXECUTE
FED1	36		PSH A		
FED2	A6 02		LDA A	2,X	COPY 3 BYTES OF PROGRAM
FED4	36		PSH A		
FED5	A6 01		LDA A	1,X	
FED7	36		PSH A		
FED8	A6 00		LDA A	0,X	OP CODE SO
FEDA	36		PSH A		SO SCRUTINIZE CAREFULLY
FEDB	8D B2		BSR	BYTCNT	
FEDD	27 75		BEQ	BSTRD	
FEDF	30		TSX		
FEE0	25 02		BCS	STEP1	
FEE2	E7 01		STA B	1,X	BRANCH OFFSET TO 2
FEE4	86 01	STEP1	LDA A	#1	
FEE6	C1 02		CMP B	#2	
FEE8	2E 06		BGT	STEP3	
FEEA	27 02		BEQ	STEP2	TWO BYTES
FEEC	A7 01		STA A	1,X	FOR ONE BYTERS
FEED	A7 02	STEP2	STA A	2,X	NOT FOR THREE BYTERS
FEF0	4F	STEP3	CLR A		NOW ADD BYTE COUNT TO PC
FEF1	EB 06		ADD B	6,X	
FEF3	A9 05		ADC A	5,X	
FEF5	A7 05		STA A	5,X	
FEF7	E7 06		STA B	6,X	

- ◆ DOES THE INSTRUCTION INVOLVE THE PC?
- ◆ IF SO THEN IT MUST BE INTERPRETED

FEF9	8D 90	STEP4	BSR	LDXU	
FEFB	A7 06		STA A	6,X	
FEFD	E7 07		STA B	7,X	UPDATE PC ON USER STACK
FEFF	C6 06		LDA B	#6	
FF01	32		PUL A		
FF02	36		PSH A		GET COPY OF OPCODE
FF03	84 CF		AND A	#\$CF	IS THIS A SUBROUTINE CALL?
FF05	81 8D		CMP A	#\$8D	
FF07	32		PUL A		
FF08	27 4E		BEQ	BSRH	
FF0A	81 6E		CMP A	#\$6E	
FF0C	27 62		BEQ	JPXH	IT IS INDEXED JUMP
FF0E	81 7E		CMP A	#\$7E	
FF10	27 65		BEQ	JMPH	IT IS EXTENDED JUMP
FF12	81 39		CMP A	#\$39	
FF14	27 69		BEQ	RTSH	IT IS RTS
FF16	81 3B		CMP A	#\$3B	
FF18	27 74		BEQ	RTIH	IT IS RTI
FF1A	81 3F		CMP A	#\$3F	
FF1C	27 76		BEQ	SWIH	IT IS SWI
FF1E	AF 06		STS	6,X	AIM USER PC AT SCRATCH AREA

SSTEP - PERFORM SINGLE STEP.

FF20 36
FF21 CE FF 30

PSH A
LDX #SSRET

REPLACE OPCODE

♦♦ SWIVE1--SET UP SWI RETURN; JUMP TO USER CODE
♦
♦ ENTRY--(X) = SWI VECTOR
♦
♦ EXIT--TO USER PROGRAM

FF24 86 7E
FF26 B7 ED F4
FF29 FF ED F5
FF2C BE ED F0
FF2F 3B

SWIVE1 LDA A #7E JUMP OP CODE
STA A SYSSWI
STX SYSSWI+1
LDS USERS
RTI

♦ THE FOLLOWING CODE IS EXECUTED AFTER EXECUTION
♦ OF AN OUT-OF-PLACE INSTRUCTION. HERE WE CHECK
♦ TO SEE IF A BRANCH HAS OCCURRED AND MODIFY
♦ THE USER'S P.C. ACCORDINGLY

FF30 30
FF31 EE 05
FF33 08
FF34 4F
FF35 5F
FF36 BC ED EE
FF39 E6 0C

SSRET TSX GET SWI HIT LOCATION INTO X
LDX 5,X
INX
CLR A
CLR B
CFX TEMP
BNE BCHNTK

♦ ADD THE BRANCH OFFSET TO THE USER PC

FF3B 09
FF3C EE 00
FF3E 09
FF3F E6 00
FF41 2A 01
FF43 43
FF44 30
FF45 EE 05
FF47 EB 01
FF49 A9 00
FF4B 30
FF4C A7 05
FF4E E7 06
FF50 09
FF51 FF ED F0
FF54 BE ED EE
FF57 39

DEX X WILL NOW POINT AT USERPC
LDX 0,X
DEX SAVED VALUE OF PC INTO X
LDA B 0,X
BPL PLUS
COM A A IS SIGN EXTENSION OF B
TSX POINT TO USERPC
LDX 5,X
BCHNTK ADD B 1,X
ADC A 0,X
TSX PLACE NEW USERPC ONTO STACK
STA A 5,X
STA B 6,X
DEX NOW X AND SP ARE EQUAL
STOX STX USERS
BSTRD LDS TEMP
RTS RETURN TO CALLING ROUTINE

♦♦ SPECIAL HANDLERS

♦♦ BSRH, JSRH - SUBROUTINE CALL HANDLERS

FF58	81 8D	BSRH	CMP A	#8D	IS IT BSR
FF5A	26 02		BNE	JSRH	
FF5C	86 5F		LDA A	#5F	THIS CONVERTS BSR'S TO BRA'S
FF5E	80 3F	JSRH	SUB A	#3F	JSR'S TO JUMPS
FF60	36		PSH A		CORRECTED OPCODE ONTO STACK
FF61	09		DEX		
FF62	09		DEX		
FF63	FF ED F0		STX	USERS	
FF66	A6 03	JSRH1	LDA A	3,X	
FF68	A7 01		STA A	1,X	MOVE USER REGISTERS
FF6A	08		INX		
FF6B	5A		DEC B		
FF6C	2A F8		BPL	JSRH1	
FF6E	20 89		BRA	STEP4	NOW EXECUTE JUMP INSTRUCT.

♦♦ JPXH - INDEXED JUMP HANDLER.

FF70	33	JPXH	PUL B		GET OFFSET
FF71	4F		CLR A		
FF72	EB 05		ADD B	5,X	
FF74	A9 04		ADC A	4,X	
FF76	8C		FCB	\$8C	CPX:: ONE BYTE BRA NEWPC
FF77	32	JMPH	PUL A		
FF78	33		PUL B		
FF79	A7 06	NEWPC	STA A	6,X	
FF7B	E7 07		STA B	7,X	
FF7D	20 D5		BRA	BSTRD	RETURN TO CALLER

♦♦ RTSH - SUBROUTINE RETURN HANDLER

FF7F	08	RTSH	INX		
FF80	08		INX		
FF81	FF ED F0		STX	USERS	NET PULL OF TWO BYTES
FF84	A6 03	RTS1	LDA A	3,X	MOVE FIVE BYTES
FF86	A7 05		STA A	5,X	
FF88	09		DEX		
FF89	5A		DEC B		
FF8A	2E F8		BGT	RTS1	
FF8C	20 C6		BRA	BSTRD	

♦♦ RTIH - RETURN FROM INTERRUPT HANDLER

FF8E	08	RTIH	INX		
FF8F	5A		DEC B		
FF90	2A FC		BPL	RTIH	
FF92	20 BD		BRA	STOX	

♦♦ SWIH - SOFTWARE INTERRUPT HANDLER

FF94	A6 07	SWIH	LDA A	7,X	
FF96	A7 00		STA A	0,X	
FF98	09		DEX		
FF99	5A		DEC B		
FF9A	2A F8		BPL	SWIH	
FF9C	8A 10		DRA A	*^00010000	SET INTERRUPT MASK
FF9E	A7 01		STA A	1,X	
FFA0	FF ED F0		STX	USERS	
FFA3	C6 FA		LDA B	*-USWI/256+256+USWI	USWI LO ORDER
FFA5	86 ED		LDA A	*USWI/256	
FFA7	20 D0		BRA	NEWPC	PATCH IN USER IRQ ADDRESS

♦♦ STRING TABLES

FFA9	0D 0A 13	PMTMSG	FCB	\$D,\$A,\$13,\$14,0,0,*,*,*,*,*+\$80
0009		PMTMSG L	EQU	*-PMTMSG
FFB2	53 39	S9MSG	FCB	'S','9'
FFB4	0D 0A 00	CRSTR	FCB	\$D,\$A,0,0,0,\$80
FFBA	53 B1	S1MSG	FCB	'S','1'+\$80

♦♦ COMMAND TABLE.

FFBC		CMDTAB	EQU	*	
0009		CMDDSP	EQU	CMDTAB-PMTMSG-PMTMSG L-1	
FFBC	52		FCB	'S'-1	SINGLE STEP
FFBD	51		FCB	'R'-1	PRINT REGISTERS
FFBE	4F		FCB	'P'-1	PUNCH MEMORY
FFBF	4C		FCB	'M'-1	DISPLAY/ALTER MEMORY
FFC0	4B		FCB	'L'-1	LOAD MEMORY
FFC1	48		FCB	'I'-1	DISPLAY INSTRUCTIONS
FFC2	46		FCB	'G'-1	GO TO USERS PROGRAM
FFC3	43		FCB	'D'-1	DUMP MEMORY
FFC4	17		FCB	\$18-1	CTL-X - DISPLAY X REGISTER
FFC5	0F		FCB	\$10-1	CTL-P - DISPLAY P REGISTER
FFC6	0B		FCB	\$C-1	CTL-L - CASSETTE LOAD
FFC7	02		FCB	3-1	CTL-C - DISPLAY C REGISTER
FFC8	01		FCB	2-1	CTL-B - DISPLAY B REGISTER
FFC9	00		FCB	1-1	CTL-A - DISPLAY A REGISTER
000E		CMDTBL	EQU	*-CMDTAB	

♦ PROCESSOR ADDRESSES FOR ABOVE COMMANDS.

FFCA	A4		FCB	EXEC-ROMBAS
FFCB	A7		FCB	REGS-ROMBAS
FFCC	1C		FCB	PNCH1-ROMBAS
FFCD	44		FCB	MEM-ROMBAS

II

AND STRIN

LES

FFCE FD
 FFCE 45
 FFD0 00
 FFD1 1B
 FFD2 88
 FFD3 86
 FFD4 F9
 FFD5 8C
 FFD6 8B
 FFD7 8A

FCB LOAD-ROMBAS
 FCB INST-ROMBAS
 FCB GO-ROMBAS
 FCB PUNCH-ROMBAS
 FCB REGX-ROMBAS
 FCB REGP-ROMBAS
 FCB LOAD1-ROMBAS
 FCB REGC-ROMBAS
 FCB REGB-ROMBAS
 FCB REGA-ROMBAS

♦♦ OP CODE TABLE - ONE BIT PER INSTRUCTION

FFD8	9C 00 3C	OPTAB	FDB	\$9C00,\$3CAF,\$4000,\$00AC
FFE0	64 12 64		FDB	\$6412,\$6412,\$6410,\$6410
FFE8	11 01 10		FDB	\$1101,\$1004,\$1000,\$1000
FFF0	11 0D 10		FDB	\$110D,\$100C,\$100C,\$100C

FFF8

ORG \$FFF8

♦♦ INTERRUPT VECTORS.

FFF8 ED F7
 FFFA ED F4
 FFFC ED FD
 FFFE FE 07
 0000

FDB	UIRQ	USER IRQ HANDLER
FDB	SYSSWI	SYSTEM SWI HANDLER
FDB	UNMI	USER NMI HANDLER
FDB	RESET	
END		

STATEMENTS =1017

FREE BYTES =22689

NO ERRORS DETECTED